Modernize your hooks



drupalday

LX

22/11/2019

About me

- Drupal developer @ Dropsolid
- Member of the board of the Portuguese Drupal Association
- Started with Drupal 5 back in 2007.



- ANNP
- Natação Pura
- Natação Sincronizada
- Pólo Aquático
- Águas Abertas
- Olímpicos

Pesquisar

This plugin is not supported

APTN REALIZA DOIS WORKSHOPS NA PÓVOA DE VARZIM

Submetido por gcomunicacao a Qua , 10/17/2007 - 18:49. ANNP

A Associação Portuguesa de Técnicos de Natação (APTN), com o apoio da Varzim Lazer, vai realizar, no dia 27 de Outubro, na Póvoa de Varzim, dois workshops.



Ler mais

JUNIORES NORTENHOS ESTAGIARAM EM RIO MAIOR

Submetido por gcomunicacao a Qua , 10/17/2007 - 18:48. Natação Pura

Os juniores Ana Ferreira (Famalicão), Joana Santos (Foca), Joana Rodrigues (FC Porto), José Gonçalves (Leixões), Cláudia Monteiro (Vilacondense), Soraia Ribeiro (Vilacondense) e Rui Costa (Vitória de Guimarães), integrados no Percurs



Ler mais

SALGUEIROS E CDUP/LIBERTY SEGUROS SEM VITÓRIAS NA TAÇA LEN

Submetido por gcomunicacao a Qua , 10/17/2007 - 18:21. Polo Aquático

Salgueiros e CDUP/Liberty terminaram a primeira eliminatória da Taça LEN sem vitórias, que se realizou, respectivamente, em Belgrado (Sérvia) e Kotor (Montenegro).



Ler mais

«SITE» REFORMULADO E COM NOVA IMAGEM

Submetido por fpereira a Ter , 10/09/2007 - 01:12.

Nova época, nova versão do site.
O «site» da Associação de Natação do Norte de Portugal (ANNP) tem, a partir de hoje, uma nova imagem e apresenta-se reformulado no que respeita aos seus conteúdos nas disciplinas de Natação Pura, Pólo Aquático e Natação Sincronizada, sem descurar as emergentes variantes de Águas Abertas e Beach Polo.

Ler mais

EVENTOS ANTERIORES

- Gala da Natação ANNP 2008
- Beach Polo 2009
- Aguas Abertas Fafe

EVENTOS

« Novembro 2019

Seg	Ter	Qua	Qui	Sex	Sáb	Dom
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	

PRÓXIMOS EVENTOS

What are Drupal Hooks?

What are Drupal hooks?

- One of the ways for modules to interact with contributed modules or Drupal core subsystems.
- A naming convention.
- Specially-named functions that a module defines

Concept

```
$hooks = ['engine', 'fuel', 'doors', 'wheels'];

$modules = ['car', 'bike'];

function bike_engine() {
 print ('Bike engine deployed<br/>');
}

function bike_fuel() {
 print ('Bike fuel deployed<br/>');
}

function car_engine() {
 print ('Car engine deployed<br/>');
}

function car_doors() {
 print ('Car doors deployed<br/>');
}
```

```
foreach($hooks as $hook) {
 foreach ($modules as $module) {
   $function = $module . '_' . $hook;
   if (function_exists($function)) {
     $function();
   }
 }
}
```

Question?
In which drupal version did the hooks appeared?



Using drupal 1.0 in 2019

Posted by fmfpereira on Friday, 11/15/2019 - 09:17 | **Announcements**

Docker is amazing!

Docker is amazing! I cant't imagine back in 2000 how long I would take to setup a LAMP stack.

[home | add a comment]

Moderate comments

Login

Username:

Password:

Login

REGISTER

Latest headlines

- Using drupal 1.0 in 2019

[more]

Modules

When developing drupal it became clear that we wanted to have a system which is as modular as possible. A modular design will provide flexibility, adaptability, and continuity which in turn allows people to customize the site to their needs and likings.

A drupal module is simply a file containing a set of routines written in PHP. When used, the module code executes entirely within the context of the site. Hence it can use all the functions and access all variables and structures of the main engine. In fact, a module is not any different from a regular PHP file: it is more of a notion that automatically leads to good design principles and a good development model. Modularity better suits the open-source development model, because otherwise you can't easily have people working in parallel without risk of interference.

The idea is to be able to run random code at given places in the engine. This random code should then be able to do whatever needed to enhance the functionality. The places where code can be executed are called "hooks" and are defined by a fixed interface.

In places where hooks are made available, the engine calls each module's exported functions. This is done by iterating through the `modules` directory where all modules must reside. Say your module is named `foo` (i.e. `modules/foo.module`) and if there was a hook called `bar`, the engine will call `foo_bar()` if this was exported by your module.

Each module has to declare an associative array named `$module` that serves as the list of hooks that a module wants to export or carry out. Each entry in the array contains the name of a hook followed by the name of the exported function.

In our above example, our associative array `$module` would look like:

```
$module = array("bar" => "foo_bar");
```

Hook name	Hook description
`admin`	If a module requires a spot in the administrator section it should implement `module_admin`. The engine will automatically add a link to the administration menus and will call `module_admin` when this link is followed. In order to make virtually any module maintainer's life easier, you don't have to worry about access rights or permissions for that matter. The engine will only allow privileged users to call exported `admin` functions.
`block`	Any module can export a block to be displayed by making the block function return an associative array of blocks. Each block in the array requires a "subject", "content", "info" and "link" field.
`cron`	Modules that require to schedule some commands to be executed on regular intervals can implement the `cron` interface: the engine will then call `module_cron` at the appropriate intervals defined by the administrator. This interface is particularly handy to implement timers or to automate certain tasks like for instance database maintenance, recalculation of settings or parameters, automatic mailings and so on.
`help`	By implementing the help hook, a module can make documentation available to the engine or to other modules.
`export`	The export hook allows a module to export data and to function as a XML/RDF/RSS backend. All you have to do in your export function is examine the array of strings containing the URI and figure out where you're at. This hook is typically used to export news headlines, but can be used to export virtually anything ranging from diary entries, reviews, rapports to entire discussions.
`page`	If a module requires its own page it should provide a function named `module_page`. The page can then be publicly accessed via `http://yourdomain.com/module.php?mod=module` which will cause the engine to invoke `module_page` in order to generate the module's page.
`user`	If a module requires to extend any of the setting pages available to individual users, it should implement the user hook. The function takes 3 parameters, resp.: `$username`, `$section` ("user", "site", "content") and `$operation` ("edit", "view", "save").

```
$module = array("help" => "fmfpereira_help",
                "admin" => "fmfpereira_admin",
                "user" => "fmfpereira_user");

// Add text on the admin page
function fmfpereira_help() {
 ?>
 <P><B>FMFPEREIRA HELO</B>: show how to implement the help hook.</P>
 <?
}

// Display text on the admin page
function fmfpereira_display() {
 print "<P><B>FMFPEREIRA</B>: custom module to show how hooks were
implemented in Drupal 1.0</P>";
}
```

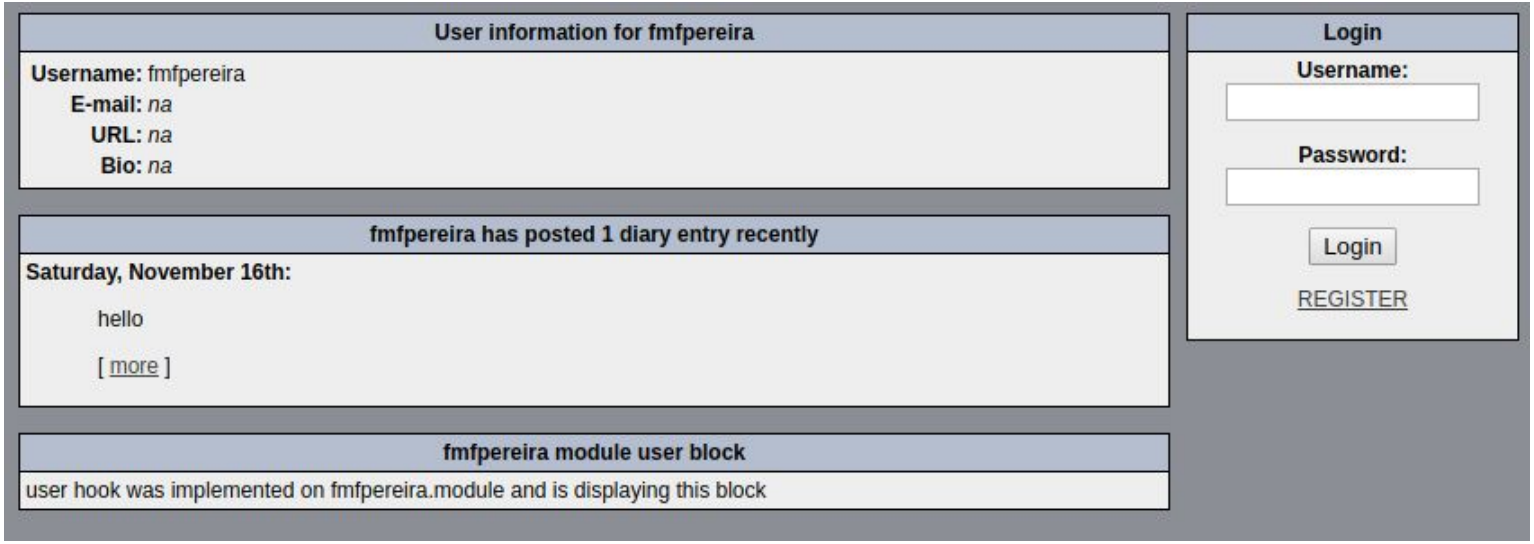
```
// Admin router
function fmfpereira_admin() {
 global $op;
 print "<SMALL><A
HREF=\"admin.php?mod=fmfpereira&op=help\">help</A></SMALL><HR>\n";
 switch ($op) {
   case "help":
     fmfpereira_help();
     break;
   default:
     fmfpereira_display();
 }
}
// Display block on the user page.
function fmfpereira_user($username, $section, $operation) {
 if ($section == "user" && $operation == "view") {
   $block["subject"] = "fmfpereira module user block";
   $block["content"] = "user hook was implemented on fmfpereira.module and is
displaying this block";
   return $block;
 }
}
```

Administration

account | affiliate-site | ban | block | box | comment | cron | diary | documentation | drupal-site | fmfpereira | headline | module | story | watchdog | home

help

FMFPEREIRA: custom module to show how hooks were implemented in Drupal 1.0



How to modernize your hooks?

```php
/**
* Implements hook_node_presave().
*/
function annp_athletes_validator_node_presave(EntityInterface $entity) {
 try {
   if ($entity->bundle() == "athletes_validator") {
     if (empty($entity->label())) {
       $entity->title = crc32(uniqid());
     }
     module_load_include("inc", "annp_athletes_validator", "annp_athletes_validator");
     if (!$entity->field_athletes_list_file->isEmpty() && !$entity->field_event_file->isEmpty()) {
       $field_athletes_list =
\Drupal::service('file_system')->realpath($entity->field_athletes_list_file->get(0)->entity->getFileUri());
       $field_event_file =
\Drupal::service('file_system')->realpath($entity->field_event_file->get(0)->entity->getFileUri());
       $event_file_content = annp_athletes_validator_get_content_from_event_file($field_event_file);
       if (!empty($event_file_content)) {
         $event_file_content_transformation =
annp_athletes_validator_apply_event_file_transformation($event_file_content);
         if (!empty($event_file_content_transformation)) {
           $athletes_list_content = annp_athletes_validator_get_content_from_athletes_list($field_athletes_list);
           if (!empty($athletes_list_content)) {
             $athletes_list_content_transformation =
annp_athletes_validator_apply_athletes_list_transformation($athletes_list_content);
             if (!empty($athletes_list_content_transformation)) {
               $result = annp_athletes_validator_validate_athletes($event_file_content_transformation,
$athletes_list_content_transformation);
               if (!empty($result["name"])) {
                 $entity->title = $result["name"];
               }
               $markup = \Drupal::theme()->render("annp_athletes_validator_result", ["validation" => $result]);
               $entity->field_athletes_validation_result->value = $markup;
               $entity->field_athletes_validation_result->format = 'full_html';
               $entity->field_athletes_validation_status->value = 1;
             }
```

Show me the juice!

Concept

Concept

- Define classes that group the implementation based on hook types.
- Implement the code in class methods.
- Use class resolver service to instantiate classes.

Class resolver

- Service that instantiate classes.
- Inject dependencies automatically if the classes implement any of the available container interfaces.
- Call the constructor if the previous step does not apply.

General types of hooks

- Hooks that answer a question.
  - Ex. hook_theme();
- Hooks that alter existing data.
  - Ex. hook_form_alter();
- Hooks that react to an action.
  - Ex. hook_user_cancel();

The code

Use case

- Two boolean fields are available on the user register form, but at least one needs to be selected.
- An extra field will be available on the user profile page to show some information placed on a twig file.

Questions?

Sources and useful information

- https://github.com/WEBSOLUTIONSHR/drupal
- https://ws.agency/blog/history-drupal
- http://www.mochasupport.com/kayako/index.php?_m=knowledgebase&_a=viewarticle&kbarticleid=542&nav=0,47
- https://hub.docker.com/r/nouphet/docker-php4/
- https://api.drupal.org/api/drupal/core%21core.api.php/group/hooks/8.7.x
- https://drupalize.me/tutorial/what-are-hooks?p=2766
- https://dri.es/drupal-hooks-vs-drupal-events
- https://www.drupal.org/project/drupal/issues/1972304
- https://www.drupal.org/project/drupal/issues/1509164
- https://www.drupal.org/node/2551893

The end